

Table of Contents

Introduction	1
RadioButtonList.....	1
Debug Assistance with Logcat.....	4
Compose: Ensuring Numeric-Only Inputs.....	5
Scaffold and Navigation with Jetpack Compose	8
Showing a List in a LazyColumn.....	11
Creating a Navigable List.....	12

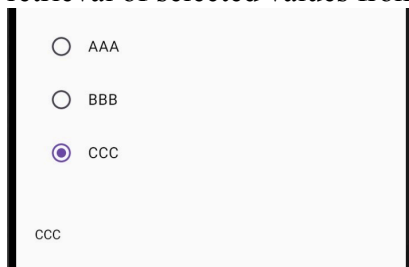
Introduction

This document will cover a few more Jetpack Compose controls, additional debugging methods as well as navigation.

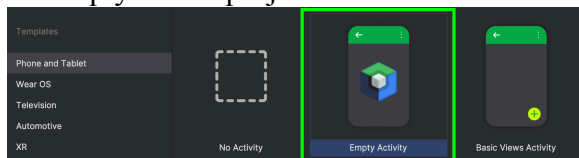
RadioButtonList

Example 1: RadioButtonList

This example demonstrates how to create a radio button list with a callback system that allows retrieval of selected values from it.



You may use an existing Kotlin project. But if you need a new Kotlin project, be sure to create an Empty Views project.



The radio button list is defined in a separate file called **MyRadioButtonList.kt**. Note that the Row is made selectable with the selectable object of the Modifier. a

Imports are required so remember to obtain the composable versions of the available libraries. You may also refer to the guide on importing Kotlin libraries from last week if needed.

Remember not to delete or overwrite the namespace (package) from the file.

```

@Composable
fun RadioButtonSingleSelection(
    rbOptions: List<String>,
    selectedRbOption:String,
    optionSelectedNotification: (String)->Unit
) {
    Log.d("*** MyRBL setup", "selectedRbOption: " + selectedRbOption)
    Column(Modifier.selectableGroup()) {
        // This loops through all items in the rbOptions list.
        rbOptions.forEach { rbOption ->

            // The row is made clickable.
            Row(
                Modifier
                    .fillMaxWidth()
                    .height(56.dp)
                    .selectable( // Selectable row is friendly for
                        // accessibility behaviour.

                        // If match, makes it the selected row.
                        selected = (rbOption == selectedRbOption),
                        onClick = {
                            Log.d("*** MyRBL clicked",
                                "selectedRbOption: " + selectedRbOption)
                                optionSelectedNotification(rbOption) },
                        role = Role.RadioButton // Making row clickable
                        // like radio button.
                    )
                    .padding(horizontal = 16.dp),
                verticalAlignment = Alignment.CenterVertically
            ) {
                // This actually will not handle clicks but it will appear to be
                // selected if it is clicked.
                RadioButton(
                    selected = (rbOption == selectedRbOption),
                    onClick = null // null is recommended for accessibility
                    // with screen readers
                )
                Text(
                    text = rbOption, // Update text with rbOption state value
                    style = MaterialTheme.typography.bodyLarge,
                    modifier = Modifier.padding(start = 16.dp)
                )
            }
        }
    }
}

```

Add this function to **MainActivity.kt**.

```
@Composable
fun ShowRadioButtonList() {
    val rbOptions = listOf("AAA", "BBB", "CCC")

    // Declare and initialize 'selectedOption' string.
    var selectedOption by remember { mutableStateOf(rbOptions[0]) }

    Column(modifier.padding(20.dp)) {
        Spacer(modifier = Modifier.height(50.dp))

        // Calling function with these three parameters assigned.
        RadioButtonSingleSelection(
            rbOptions = rbOptions,
            selectedRbOption = selectedOption, // Initialize radio button list

            // Callback to radiobutton list.
            optionSelectedNotification = {
                // Receive value back from radio button list.
                // 'it' contains object that is passed from publisher.
                Log.d("*** MainActivity CB", "it value: " + it)
                selectedOption = it
            }
        )
        Spacer(modifier = Modifier.height(50.dp))
        Text(selectedOption)
    }
}
```

Next, replace the contents of `setContent()` with:

```
ShowRadioButtonList()
```

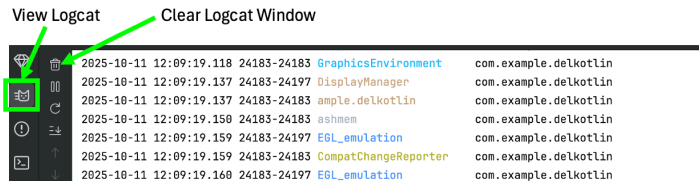
Exercise 1 (1 mark)

In your own words, explain what each of the parameters of the `RadioButtonSingleSelection` function do?

Is the `selectedRbOption` parameter of `RadioButtonSingleSelection()` function mutable? Why or why not?

Debug Assistance with Logcat

While I always advocate stepping through your code with the debugger, logcat will help you to print out variables values in real-time. This is useful since a console is not available for logging by default. To access Logcat go to **View | Tool Windows | Logcat**. Click the Logcat icon to view it can be found at the bottom left of the IDE. The Logcat window can sometimes be overcrowded with text so you may also clear it before starting a log session.



Instead of using print statements, the `Log.d()` instruction is used to print output. Two parameters 'tag' and 'msg' are required. Logcat prints out a lot of extra activity information so I prefix my tags with '***' so I can quickly find my printed output.

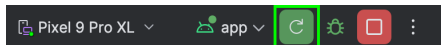
```
optionSelectedNotification = {  
    // Receive value back from radio button list.  
    // 'it' contains object that is passed from publisher.  
    Log.d( tag = "*** MainActivity CB", msg = "it value: " + it)  
    selectedOption = it  
}
```

The output from the instruction above appears in logcat as:

```
*** MainActivity CB      com.example.delkotlin      D      it value: BBB
```

Exercise 2 (2 marks)

Starting with the solution for Example 1, open the logcat window and clear it by clicking the trash can icon. Then re-run the app.



Show a screenshot of the three lines of custom logged output which appear together after clicking on a new option.

Explain how the value of `selectedRbOption` is set during the sequence of these three logged events.

Is the radio button list re-rendered after an item within it is changed?

Exercise 3 (1 mark)

Create a radio button option with your first name in it. Then show a screenshot of the logcat output when clicking on the option with your name in it.

Compose: Ensuring Numeric-Only Inputs

Example 2: Enabling Numeric Only Inputs

This example ensures that only numeric data can be entered in the OutlinedTextField control:



To build this example, create a file called **MyNumericInput.kt** and place this code in it. Remember not to delete or overwrite the package name.

```

data class Message(val author: String, val body: String)

val integerChars = '0'..'9'
fun isNumber(input: String): Boolean {
    var dotOccurred = 0
    return input.all { it in integerChars || it == '.' && dotOccurred++ < 1 }
}
fun isInteger(input: String) = input.all { it in integerChars }

@Composable
fun decimalFieldInput() {
    // changeable text.
    var text by remember { mutableStateOf("") }
    val pattern = "^\\d+\\$"

    OutlinedTextField(
        value = text,
        onValueChange = {
            if (isNumber(it)) {
                text = it
            }
        },
        keyboardOptions = KeyboardOptions(keyboardType = KeyboardType.Number),
        label = { Text("Name") }
    )
}

@Composable
fun MessageCard(msg: Message) {
    var text by remember { mutableStateOf("") }
    val modifier: Modifier = Modifier
    Column(modifier.fillMaxSize()
        .padding(24.dp),
        verticalArrangement = Arrangement.spacedBy(16.dp)
    ) {
        // Adds space above first control.
        Spacer(modifier = modifier.height(40.dp))
        Text(text = msg.author, modifier.padding(20.dp), fontSize=20.sp)
        Text(text = msg.body)
        decimalFieldInput()
    }
}

```

Next, in **MainActivity.kt** replace the onCreate event handler with this version to render the new numeric input.

```

override fun onCreate(savedInstanceState: Bundle?) {
    super.onCreate(savedInstanceState)
    enableEdgeToEdge()
    setContent {
        MessageCard(Message("Pat", "Jetpack Compose!"))
    }
}

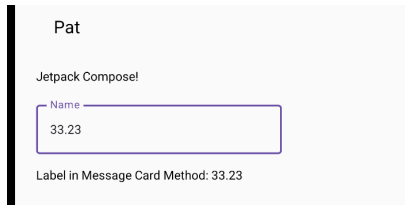
```

To encourage code reuse, the number checking functions **highlighted above** can be moved to a separate Kotlin file.

Exercise 4 (3 marks)

Modify the text input from Example 2 so `decimalFieldInput()` sends the text value inside it to the `MessageCard()` method. Hint: Pass a function reference to `decimalFieldInput()`.

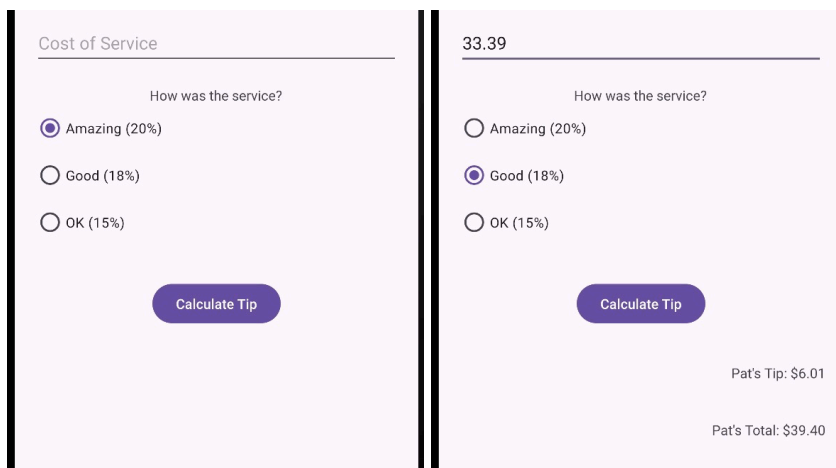
Figure 1: Exercise 4 sample



Show a screenshot of the output screen, including the output label, like in Figure 1 but with your name on it.

Exercise 5 (8 marks)

Using Kotlin, create a tip calculator like the one below. The screen (activity) when first loaded appears as shown on the left. The image on the right shows the activity after the cost of the meal has been entered, the user can then select a tip % and click the button to calculate it.

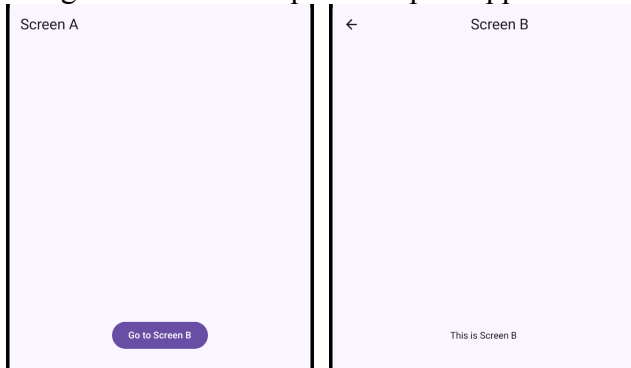


Show a screenshot of the interface when done. But sure that your name appears as shown at the bottom right.

Scaffold and Navigation with Jetpack Compose

Example 3: Scaffold and Navigation

Similar to Flutter, Kotlin offers components to implement scaffold layouts and to enable navigation. This example sets up an app with routes for screen A and screen B.



MainActivity.kt

```

class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContent { MyApp() }
    }
}

@Composable
fun MyApp() {
    val navController = rememberNavController()
    // Define routes
    NavHost(navController, startDestination = "screenA") {
        composable("screenA") { ScreenA(navController) }
        composable("screenB") { ScreenB(navController) }
    }
}

@OptIn(ExperimentalMaterial3Api::class)
@Composable
fun ScreenA(navController: NavController) {
    Scaffold(
        topBar = {
            TopAppBar(
                title = { Text("Screen A") }
            )
        }
    ) { // In a scaffold component, Kotlin sets the padding for you.
        padding ->
        Column(
            modifier = Modifier
                .fillMaxSize()
                .padding(padding),
            horizontalAlignment = Alignment.CenterHorizontally,
            verticalArrangement = Arrangement.Center
        ) {
            Button(onClick = { navController.navigate("screenB") }) {
                Text("Go to Screen B")
            }
        }
    }
}

@OptIn(ExperimentalMaterial3Api::class)
@Composable
fun ScreenB(navController: NavController) {
    Scaffold(
        topBar = {
            TopAppBar(

```

Exercise 6 (1 mark)

What is the purpose of the NavHost in this app?

Exercise 7 (1 mark)

How does `navController.navigate("screenB")` differ from `navController.popBackStack()`?

Exercise 8 (1 mark)

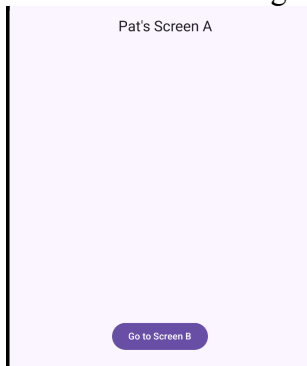
What parameter does each screen function receive?

Exercise 9 (1 mark)

What is the purpose of a Scaffold component in Kotlin?

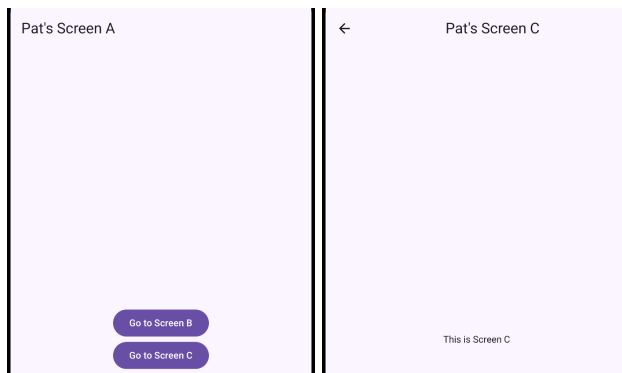
Exercise 10 (1 mark)

In Kotlin, center the text for the screen A title but change it to your first name. Show a screenshot of the text after changing the name. It should look like the following but with your name in it.



Exercise 11 (2 marks)

Add a second button to go to screen C. Add a `screenC` function. Modify the route in the screen C button so the app navigates to screen when it is clicked.



Show screenshots of screen A with the two buttons and your screen C. Be sure your name appears in these screenshots rather than mine.

Show the code that you used to implement your two buttons.

Exercise 12 (2 marks)

Figure out how to pass your name as a parameter to screen B. Here is a hint:

```
composable("screenB") {
    ScreenB(navController=navController, fname="Pat")
}
```

Include your name in the output that appears on screen B. Show your code for the screen B function after enabling this change.

Showing a List in a LazyColumn

The LazyColumn and sibling LazyRow provide a memory-friendly just-in-time rendering approach which is scrollable.

Example 4: LazyColumn

This example demonstrates how to display a list in a LazyColumn structure. The list in this example is not very useful but the demonstration offers a simple view of how to make the LazyColumn work. The LazyColumn will be very important later in several examples that follow.

Person 1
Person 2
Person 3

In a new Empty Activity project, after where the theme is defined for your project, add in a Surface component which calls ShowList() to display the LazyColumn list. Also add the ShowList() function which displays the list.

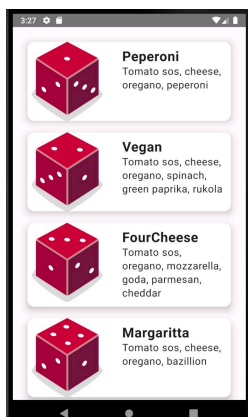
```
class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContent {
            Surface(color= MaterialTheme.colorScheme.background) {
                ShowList()
            }
        }
    }
}

@Composable
fun ShowList() {
    val names = listOf("Person 1", "Person 2", "Person 3")
    LazyColumn(
        modifier = Modifier.padding(top = 70.dp) // 🖱️ adds space above list
    ) {
        items(names) { name ->
            Text(name, modifier = Modifier.padding(8.dp))
            HorizontalDivider()
        }
    }
}
```

Creating a Navigable List

Example 5: Creating a Navigable List

This example demonstrates how to create a scrollable and navigable LazyColumn list for a pizza menu. Each menu item can be clicked to navigate to a corresponding detail page.



Starting with a new Empty Activity project, add a **MainScreen.kt** file. The `MainScreen()` function defines the `LazyColumn` list format. The `ColumnItem()` function defines an object for formatting the detail within each block of the list.

MainScreen.kt

```

@Composable
fun MainScreen(
    imageIds: Array<Int>,
    names: Array<String>,
    ingredients: Array<String>,
    navController: NavController,
    modifier: Modifier = Modifier
) {
    // The LazyColumn stores build the list.
    LazyColumn(
        contentPadding = PaddingValues(top = 80.dp, start = 16.dp,
                                         end = 16.dp, bottom = 16.dp)) {

        val itemCount = imageIds.size

        // items of the LazyColumn receives the list count as a parameter.
        // requires
        // import androidx.compose.foundation.lazy.items
        items(itemCount) {
            // A call to ColumnItem passes the list data.
            ColumnItem(
                modifier,
                painter = imageIds,
                title = names,
                ingredients = ingredients,
                itemIndex = it,
                navController = navController
            )
        }
    }
}

// Provides formatting for items in the list.
@Composable
fun ColumnItem(
    modifier: Modifier,
    painter: Array<Int>,
    title: Array<String>,
    ingredients: Array<String>,
    itemIndex: Int,
    navController: NavController
) {
    Card(
        modifier
            .padding(10.dp)
            .wrapContentSize()
            .clickable {

```

DetailScreen defines the detail page that displays a description for a single list item.

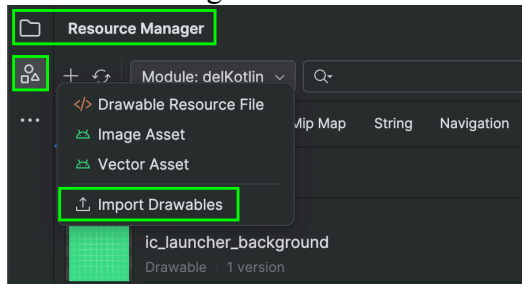
DetailScreen

```

@Composable
fun DetailScreen(
    navController: NavController,
    modifier: Modifier = Modifier,
    photos: Array<Int>,
    names: Array<String>,
    ingredients: Array<String>,
    itemIndex: Int?,
) {
    Scaffold(
        topBar = {
            TopAppBar(
                title = {
                    Box(
                        modifier = Modifier.fillMaxWidth(),
                        contentAlignment = Alignment.Center
                    ) {
                        Text("Details")
                    }
                },
                navigationIcon = {
                    IconButton(onClick = { navController.popBackStack() }) {
                        Icon(
                            imageVector = Icons.Filled.ArrowBack,
                            contentDescription = "Back"
                        )
                    }
                }
            )
        }
    ) { padding ->
        Column(
            modifier = modifier
                .fillMaxSize()
                .padding(padding)
                .padding(25.dp),
            verticalArrangement = Arrangement.spacedBy(20.dp)
        ) {
            Row(
                modifier = Modifier.fillMaxWidth(),
                horizontalArrangement = Arrangement.Center
            ) {
                Image(
                    painter = painterResource(id = photos[itemIndex!!]),
                    contentDescription = names[itemIndex]
                )
            }
        }
    }
}

```

Inside **setContent()** in MainActivity.kt within onCreate() add the following code. The dice images are available in the day 2 lab folder. Each image can be added to the project with the Resource Manager.



This code belongs in setContent to define the main listing route, *MainScreen*, and the detail screen route, *DetailScreen*.

```

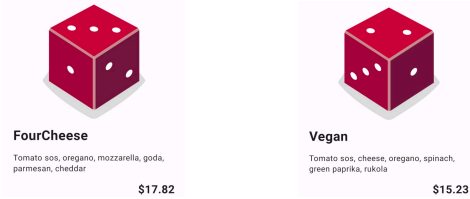
Surface(
    modifier = Modifier.fillMaxSize(),
    color = MaterialTheme.colorScheme.background
) {
    // Define equally sized arrays to store images, titles, and descriptions.
    val imageIds = arrayOf(
        R.drawable.dice_1,
        R.drawable.dice_2,
        R.drawable.dice_3,
        R.drawable.dice_4,
        R.drawable.dice_5,
        R.drawable.dice_6
    )
    val names = arrayOf(
        "Peperoni",
        "Vegan",
        "FourCheese",
        "Margaritta",
        "American",
        "Mexican"
    )
    val ingredients = arrayOf(
        "Tomato sos, cheese, oregano, peperoni",
        "Tomato sos, cheese, oregano, spinach, green paprika, rukola",
        "Tomato sos, oregano, mozzarella, goda, parmesan, cheddar",
        "Tomato sos, cheese, oregano, bazillion",
        "Tomato sos, cheese, oregano, green paprika, red beans",
        "Tomato sos, cheese, oregano, corn, jalapeno, chicken",
    )

    val navController = rememberNavController()
    NavHost(navController = navController, startDestination = "MainScreen") {
        // Define equally sized
        composable(route = "MainScreen") {
            MainScreen(imageIds, names, ingredients, navController)
        }
        composable(route = "DetailScreen/{index}",
            arguments = listOf(
                navArgument(name = "index") {
                    type = NavType.IntType
                }
            )
        ) { index->
            DetailScreen(
                photos = imageIds,
                names = names,
                ingredients = ingredients,
                itemIndex = index.arguments?.getInt("index"),
            )
        }
    }
}

```

Exercise 13 (4 marks)

Define six prices as strings where the other data lists are defined. Modify the code to pass the prices to the detail view and to display them there so it looks like the following sample displays:



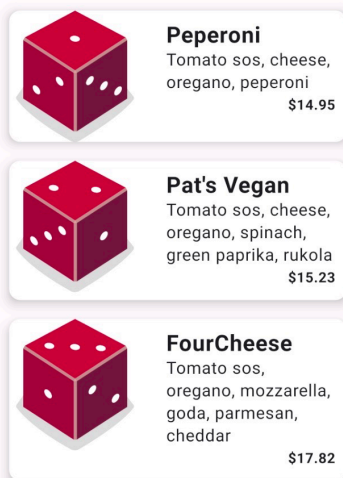
Show your adjusted code as text for **MainActivity** here:

Show your adjusted code as text for **DetailScreen** here:

Show a screenshot of detail screen with a pizza that has your first name in the title:

Exercise 14 (4 marks)

Figure out how to add a price so it appears as follows in the main screen list:



Show a screenshot of detail screenshot with at least two pizza's which have different prices in the list. One pizza must have your name in the title.